

AI Impact on Jobs: Salary, Productivity & Employment Analysis

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import seaborn as sns

import warnings
warnings.filterwarnings('ignore')
```

!gdown 1ZPAttEMRLSYPH8NZV53TFKp2WmM5lMr

Downloading...
From: <https://drive.google.com/uc?id=1ZPAttEMRLSYPH8NZV53TFKp2WmM5lMr>
To: /content/ai_job_impact.csv
100% 197k/197k [00:00<00:00, 38.5MB/s]

```
df = pd.read_csv("ai_job_impact.csv")
```

Start coding or generate with AI.

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Employee_ID                          2000 non-null   object
1   Age                                   2000 non-null   int64
2   Gender                               2000 non-null   object
3   Education_Level                      2000 non-null   object
4   Industry                             2000 non-null   object
5   Job_Role                             2000 non-null   object
6   Years_Experience                     2000 non-null   int64
7   AI_Adoption_Level                   2000 non-null   object
8   Automation_Risk                     2000 non-null   object
9   Upskilling_Required                 2000 non-null   object
10  Salary_Before_AI                    2000 non-null   int64
11  Salary_After_AI                     2000 non-null   int64
12  Job_Status                          2000 non-null   object
13  Work_Hours_Per_Week                 2000 non-null   int64
14  Remote_Work                         2000 non-null   object
15  Job_Satisfaction                    2000 non-null   int64
16  Productivity_Change_%               2000 non-null   float64
dtypes: float64(1), int64(6), object(10)
memory usage: 265.8+ KB
```

df.describe()

	Age	Years_Experience	Salary_Before_AI	Salary_After_AI	Work_Hours_Per_Week	Job_Satisfaction	Productivity
count	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000	2000.000000
mean	40.558000	16.663500	73942.072500	78428.642500	44.85100	6.020500	6.020500
std	10.786418	10.746675	26055.823793	29351.599013	5.71254	2.006263	2.006263
min	22.000000	0.000000	30036.000000	24447.000000	35.00000	3.000000	3.000000
25%	32.000000	8.000000	51665.500000	54086.750000	40.00000	4.000000	4.000000
50%	40.000000	16.000000	74620.000000	76820.500000	45.00000	6.000000	6.000000
75%	50.000000	26.000000	95418.250000	100730.250000	50.00000	8.000000	8.000000
max	59.000000	37.000000	119976.000000	161745.000000	54.00000	9.000000	9.000000

```
df.columns
```

```
Index(['Employee_ID', 'Age', 'Gender', 'Education_Level', 'Industry',
       'Job_Role', 'Years_Experience', 'AI_Adoption_Level', 'Automation_Risk',
       'Upskilling_Required', 'Salary_Before_AI', 'Salary_After_AI',
       'Job_Status', 'Work_Hours_Per_Week', 'Remote_Work', 'Job_Satisfaction',
       'Productivity_Change_%'],
      dtype='object')
```

Adding additional columns

```
df['salary_change'] = df['Salary_After_AI'] - df['Salary_Before_AI']

df['salary_change_pct'] = df['salary_change']/df['Salary_Before_AI'] *100

df['salary_change_bin'] = np.where(
    df['salary_change'] > 0, 'Increase',
    np.where(df['salary_change'] < 0, 'Decrease', 'No Change')
)

df['Years_Experience_bin'] = pd.cut( df["Years_Experience"], bins = [ 0,5,15,50 ] , labels= [ 'low' , 'Medium' , 'High'] )
```

df.head()

	Employee_ID	Age	Gender	Education_Level	Industry	Job_Role	Years_Experience	AI_Adoption_Level	Automation_Risk	Unemployment_Rate
0	E0001	50	Female	Bachelor	Marketing	Content Creator	26	High	High	12.5
1	E0002	45	Male	High School	Manufacturing	Quality Inspector	19	Low	Low	10.2
2	E0003	51	Female	Master	IT	DevOps Engineer	28	Medium	Medium	8.7
3	E0004	48	Male	PhD	Education	Teacher	24	Medium	Medium	9.1
4	E0005	24	Male	Bachelor	Healthcare	Doctor	0	High	Medium	7.3

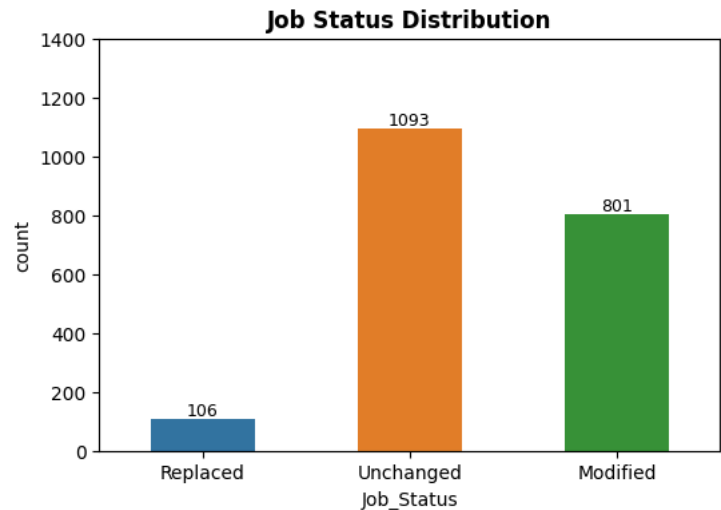
5 rows × 21 columns

Job Status Distribution

```
plt.figure(figsize=(6,4))
ax = sns.countplot(data=df, x='Job_Status' , width = 0.5 , hue = 'Job_Status' )
plt.title('Job Status Distribution' , fontweight = 'bold')

for i in ax.containers:
    ax.bar_label(i , fontsize = 9)
plt.ylim(0 , 1400)

plt.show()
```



AI Impact on Job Status Low Job Replacement (5%) Only a small fraction of jobs are replaced, indicating AI is not causing massive layoffs.

Majority Jobs Remain Stable (55%) Most roles are unchanged, showing gradual and controlled AI adoption.

Significant Role Transformation (40%) A large portion of jobs are modified, highlighting that AI is reshaping responsibilities.

AI as Augmentation, Not Replacement Overall trend suggests AI is enhancing human roles rather than eliminating them.

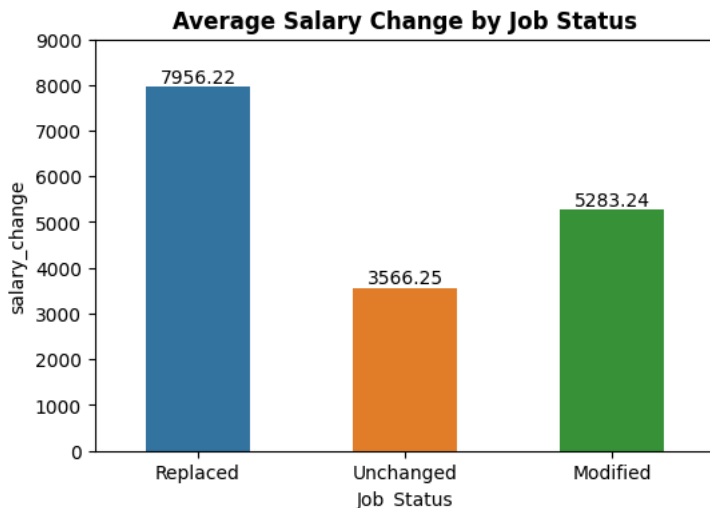
```
# from google.colab import files

# df.to_csv("ai_jobs_analysis.csv" , index = None)
```

```
plt.figure(figsize=(6,4))
ax=sns.barplot(data=df, x='Job_Status', y='salary_change' , hue = 'Job_Status' , errorbar= None , width= 0.5)

for i in ax.containers:
    ax.bar_label(i , fontsize = 10)

plt.title('Average Salary Change by Job Status' , fontweight = 'bold')
plt.ylim(0 , 9000)
plt.show()
```



Highest Salary Increase in Replaced Roles Jobs classified as replaced show the highest average salary change (~₹7,956), indicating that transitions often lead to better-paying roles or opportunities. Moderate Gains in Modified Roles

Modified jobs also experience strong salary growth (~ ₹5,283), suggesting that adapting to AI (role evolution) can be financially beneficial. Lowest Growth in Unchanged Roles

Unchanged jobs have the smallest increase (~₹3,566), showing limited financial upside when roles remain static.

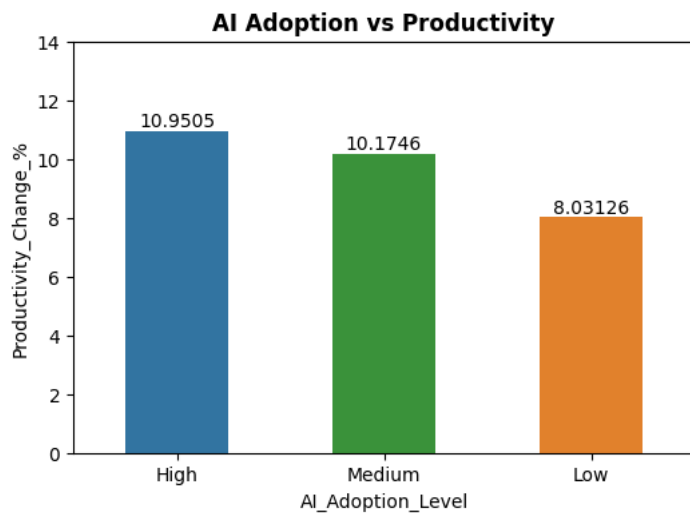
Upskilling & Transition Drive Better Outcomes Overall, roles impacted by AI (replaced/modified) tend to see higher salary growth, highlighting the importance of adaptability and skill upgrades.

```
order = df.groupby('AI_Adoption_Level')['Productivity_Change_%'].mean().sort_values( ascending= False).index

plt.figure(figsize=(6,4))
ax=sns.barplot(data=df, x='AI_Adoption_Level', y='Productivity_Change_%' , hue = 'AI_Adoption_Level' , errorbar= None , width= 0.5)

for i in ax.containers:
    ax.bar_label(i , fontsize = 10)

plt.title('AI Adoption vs Productivity' , fontweight = 'bold')
plt.ylim(0 , 14)
plt.show()
```



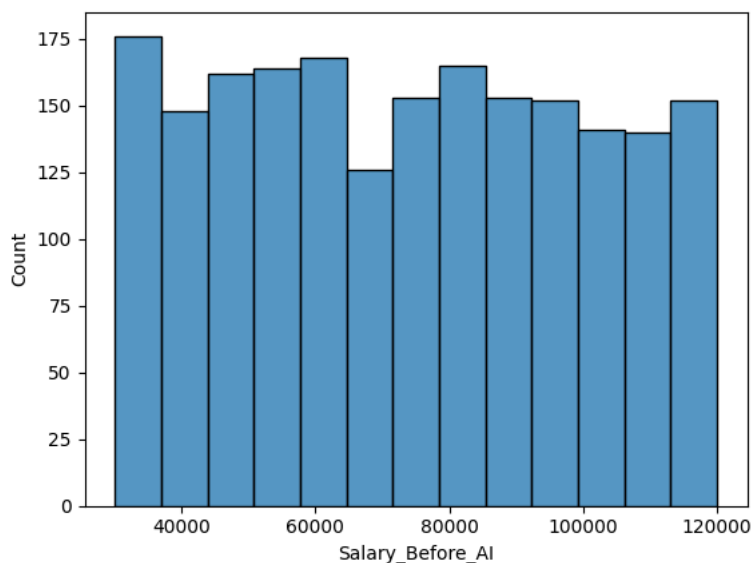
Higher AI Adoption → Higher Productivity Employees with high AI adoption show the highest productivity gain (~10.95%), indicating strong positive impact of AI tools.

Consistent Improvement Across Levels Even medium adoption delivers similar gains (~10.17%), showing that partial adoption still adds value.

Low Adoption Limits Performance Low AI adoption has the lowest productivity increase (~8.03%), highlighting missed efficiency opportunities.

Diminishing Gap at Higher Levels The difference between high and medium adoption is small, suggesting early adoption brings most benefits, while full adoption gives incremental gains.

```
sns.histplot( data = df , x = 'Salary_Before_AI' )  
plt.show()
```



```
plt.figure(figsize=(10,4))  
  
plt.subplot(1,2,1)  
sns.histplot(df['Salary_Before_AI'], color='blue', kde=True)  
plt.title('Before AI')  
  
plt.subplot(1,2,2)  
sns.histplot(df['Salary_After_AI'], color='red', kde=True)  
plt.title('After AI')  
  
plt.tight_layout()  
plt.show()
```

